中国研究生操作系统开源创新大赛

成员分工及主要贡献说明

作品名称：面向智能机器人的 AIOS

参赛队伍：Xenoblade

完成日期：2026 年 10 月

队员分工

本队共三名成员，按子系统分工，各自负责所辖模块的设计、实现、板级测量与复现。队员一承担 视觉推理与驱动协同的主体工作：SG2002 上 StarryOS 的端到端视觉推理链路，UVC、JPU、VPSS、 ION 与 TPU 驱动的实现与优化，YOLOv8 网球检测与机器人拾球闭环，以及全链路插桩与 StarryOS/Linux 同口径基线的对比验证。

其余两位成员的工作按“谁拥有哪条线的结论”切分。内核优化这条线体量最大，因此拆成两半：队员二负责改动的实现、正确性修复与功能使能，队员三负责指标、基准与验收口径；除此之外，队员三负责实时小核这条线，队员二负责板级 I/O 与上板工作流。三人的详细交付物见下表。

成员	负责模块与交付物
队员一	**视觉推理与驱动协同**。 围绕 SG2002 构建 StarryOS 端到端视觉推理链路，参与并完成 **UVC、JPU、VPSS、ION 与 TPU 驱动的实现和优化**。UVC 后台异步采集、双缓冲最新帧发布、 帧序号同步与一帧背压，避免历史帧积压；JPU 硬件解码 MJPEG 4:2:2，YUV422P 三平面直接 写入 ION 物理缓冲，补齐 JPEG 完整性校验、有界重试、BBC 256 B 页边界预取和异常复位； VPSS 扩展 YUV422P 三平面输入 ABI，在单次离线任务中完成 CSC、letterbox、缩放及 RGB planar/NCHW 输出，修复完成 IRQ 后约 43 ms 的调度长尾，使 VPSS 墙钟降至约 1.64 ms。 **TPU 驱动**。 对齐 SOPHGO/CVITEK 官方 ioctl、ION fd 解析、`WAIT_DMABUF` 返回语义和 任务完成状态，补齐 CV181x 时钟、复位与任务生命周期管理（TPU 700 MHz、FAB 900 MHz）； 建立 TDMA-only/compute 任务分类及 `submit_to_run`、`fire_to_irq`、`irq_to_resume`、 `done_to_user` 四段计时，采用同步直通执行消除提交线程与 worker 的重复调度交接， 并通过 TDMA IRQ 队首唤醒和 2 ms 兜底轮询修复中断一定时器竞态，将 TDMA 输入搬运稳定至 约 1.11 ms。 **用户态与机器人闭环**。 完成 YOLOv8 网球检测、红色容器识别、目标选择、二次曲线减速、 近球微调、机械臂抓取、容器抵近、松球、后退和再次搜索状态机，实现多球连续自主回收。 **测评与验证**。 建立摄像头、JPU、VPSS、TDMA、TPU、后处理全链路插桩和 StarryOS/Linux 同口径基线，StarryOS 单帧视觉时延 53.024 ms、等效处理速率 18.86 FPS、 平均置信度 93.13%，较 Linux 基线 54.230 ms 降低 2.22%，完成三球任务与长时间 稳定性验证。
队员二	**内核优化的实现与缺陷修复**。 以“每次操作都要付的固定成本”为主线，负责内核各子系统 改动的实现与正确性验证，与队员三共同完成与 Linux 基线的逐项对照。 返回路径上定位到每次返回用户态都执行一次 `fence.i`（2.17 µs），把默认行为反转为“不刷”、只在四类事件上刷一次，`getpid` 由 10 560 ns 降到 1 631 ns，与 Linux 的差距 从 25.0× 收到 3.9×；进程一侧去掉 COW 复制里每个叶子的第二次页表走查、把所有者发布 去重由 $O(n^2)$ 改为 $O(n \log n)$、地址空间退休不再逐页 `unmap`（3.57 ms → 0.41 ms）； `execve` 只物化带写权限的段并按 run 批量 `pin`，exec 内核侧由 32.3 ms 降到 4.5 ms； 缺页一侧修掉 COW 页索引每次插入全量重建造成的 $O(n^2)$，把内核导出的 `memcpy`/`memmove`/`memset`/`memcmp` 从逐字节循环改成字长批量，并实现 64 KiB 窗口 顺序预取、窗内帧整块分配与子租约、索引按 run 批量登记——64 MiB 默认路径的每页

	成本 由 1 302 123 ns 降到 44 120 ns，与 Linux 的差距从 165× 收到 5.6×，且成本不再随 区域大小增长。 文件与存储一侧，缓存命中读去掉暂存中转、用户内存准备整段化、延迟失效路径不再逐页 刷 TLB，写回按连续页合并下发、页缓存上限由 512 页提到 4096 页、`close` 不再无条件 同步写访问时间：ext4 `open`+`read`(1 KiB)+`close` 由 10 320 µs 降到 454 µs，8 MiB 顺序写 + `fsync` 由 7 961 ms 降到 931 ms，页缓存顺序写由 2.41 MiB/s 提到 42.43 MiB/s（同板 Linux 为 34 MiB/s）；网络初始化里固定 2 s 的阻塞等待改成带截止 时间的轮询（上限 300 ms），上电到 shell 可交互由 12.3 s 降到 10.8 s。 **真实缺陷的修复与功能使能**。 修掉三处缺陷：`memmove` 反向路径在 `i == 0` 时地址 下溢的未定义行为（用户进程偶发丢输出并以 `SIGSEGV` 退出）、COW 页索引里过期墓碑 遮蔽同址重分配帧的身份、使 `fork`/COW 在帧回收后走错分支，以及 `ax-net` 路径解析 在自旋锁内与全局同步写回撞车引起的 EIO/panic。补齐 strace、fstrace、FAT 与 dmesg 能力，恢复无线链路在长传输下的可用性。同族里“整段插入”的第一版实现曾让板子在启动 过程中完全无输出、连软件复位都发不进去，重做后确立三条安全设计——整段插入全有或 全无、遇到空洞或区间内墓碑退回逐页登记、容量按整段预留以免在关中断的临界区里分配 内存；这次回归作为负结果写入报告。 **板级 I/O 与上板工作流**。 建立“软复位 → U-Boot UMS → `mcopy`/`debugfs` 写入 → `fatload`/`bootm`”的无人值守上板流程，配置串口命令执行器、启动三锚点探针与 Linux 侧登录窗口抓取工具；给出单路控制台下三路候选 UART 焊盘的取舍、UMS 退出后节点残留 与空间统计滞后的两条边界、软复位只复位 SoC 不复位执行机构，以及 PWM 的四处语义 落差。
队员三	**实时小核启用**。 把厂商固件里“存在但未被使用”的 RISC-V C906@700 MHz 协处理器 （C906L）做成可被 StarryOS 管理与使用的第二个可编程执行体。 核间通道一侧实现 `ap_mailbox`（`0x01900000`，8 槽 × 8 字节 `cmdqu` 信封）驱动： 中断上下文只用 IRQ-save 自旋锁与固定容量环形队列，等待路径按“硬中断 → 服务线程 → 等待队列”三段式实现；另开辟 1 MiB 非缓存共享区并定义 v3 布局（头部按写入方拆 cache line、双 64 槽环 + CRC32），对用户态注册 `/dev/cvi-rtos-cmdqu`，支持指定 应答命令号的 `EXCHANGE` 与 `poll`/`epoll` 语义。跨核往返延迟为 min 53 / p50 54 / max 413 µs，旧轮询实现的下界为 1000 µs。 底盘链路接管分三步落地、每步各设判据：UART1 独立握手（命令 `0x61`，判据是拔线冻结、插回恢复）、协议自检（mode=2，判据是全程零速度指令）、周期服务与心跳看门狗 （`0x62` 进入 100 ms 周期、`0x64` 设 500 ms 心跳窗口），另加不依赖新增硬件的堵转 与碰撞保护（kind=10）；机械臂舵机由 UART2 直接驱动（命令 `0x66` + kind=11）。确定性一侧给出同机、同负载、同链路的 A/B：大核直控在 3 个计算负载下 p99 恶化到 +1.54 ms、最坏 +1.75 ms，小核接管在全部负载下 min = mean = max、样本 100% 落在 期望 tick 刻度；大核停喂心跳约 500 ms 后小核自主停车，小核被停掉后大核侧的只读 判据在 2 s 内确认并抢回链路补发停车序列。固件与构建在 LicheeRV-Nano-Build 侧维护， 补齐小核时钟门控与命令上报路径。 **内核优化的指标与基准**。 与队员二共同完成“对比 Linux 的完整基准结果”：建立内核侧 分段计数出口 `/proc/fault_attrib` 与“工作负载前后快照之差”的取数口径，自研 `syscost2`、`sa1`、`sp2`、`fk1`/`fk2`、`faultcost`、`faultattrib`、`popcost`、 `touch64`、`bloat`、`cowtest`、 `read_bench`、`read_syscall`、`rp1`/`rp2`、`mmc`、 `block-rw-bench` 等探针；确立冷启

	动受控 A/B 口径（同一次上电、交替顺序、各测两遍 取均值）与“探针本身有代价”的口径（热路径每次多 12 次取时间戳，约 1.2 µs），维护 Linux／优化前／优化后三列对照；建立宿主单测、板端功能、数据正确性、内存不泄漏、 TLB 一致性与压力测试等验收门，凡动到对外语义的改动都保留运行期回退开关。

表 1: 三名成员按模块与交付物的分工

视觉链路的结论建立在同口径基线上: 摄像头取流、JPU 解码、VPSS 预处理、TDMA 搬运、TPU 前向与 后处理六段各自插桩计时, 两侧使用同一份符合 Linux ABI 的可执行文件与同一模型, 因此同一负载在 StarryOS 与 Linux 上呈现的差别只来自操作系统本身。VPSS 完成中断后的调度长尾与 TPU 的 中断—定时器竞态都是先由分段时间戳定位、再落到驱动改动，修复前后的对比数据与复现命令另见测试文档。

三条工作线在两处交汇。其一, 视觉链路与内核优化共用同一条上板通路: 同一张 SD 卡、同一份 FIT 映像、同一套串口与 UMS 工作流, 是所有基线能够反复取得的共同前提。其二, 两侧的结论共用 同一套取数方法: 内核侧由 `/proc/fault_attrib` 的计数器按“工作负载前后快照之差”给出分段 数字，视觉侧的分段探针与 Linux 基线在同一次上电周期内交替采集，因此两边的“优化前／优化后／ Linux”三列都可以在板上由脚本复现。各条改动与数据的可核对入口（提交号、探针与复现命令） 见《项目说明书》的“代码原创说明”与“测试说明”两节。